Code Formatting and Linter Suppressions

Independent research report | Public draft, November 2026

1. Introduction

A `noqa` comment affects linting through its position in the source. Joining two lines can put more code under the same comment; splitting a statement can leave part of it uncovered. Neither change requires a different parsed program structure. This study asks when Black and Ruff produce such changes for three selected Ruff rules.

For example, `# noqa: F821` suppresses undefined-name warnings within its scope. If formatting moves that comment beside two names instead of one, a warning may disappear from ordinary lint output while the name that caused it is still present. The reverse can happen when a formatter separates statements.

The comparison follows warning occurrences, rather than just counting them. Equal counts can conceal a swap in visibility, and changed counts can reflect different underlying diagnostics. Each source pair therefore goes through four lint passes and a correspondence check before a change is attributed to suppression.

Controlled inputs exercise particular layouts and directive placements. A separate sample of repository files checks for changes in naturally occurring source. The two datasets answer different questions and are reported separately.

2. Related Work

Hu et al. [1] studied suppressions across project histories. Their treatment of line splitting is relevant here because a changed warning count need not mean a new warning. Li and Yang [2] studied correspondence across revisions, a broader setting than the AST-stable, rule-restricted pairs used in this experiment. McKeeman's differential-testing account [3] supplies the comparison principle: disagreement identifies a case to investigate.

Formatter documentation already describes this interaction. Black's FAQ discusses moved linter comments [4], and Ruff gives a list-collapse example that broadens `noqa` coverage [5]. This experiment measures that known possibility under fixed settings, with saved warning identities and a separate repository sample. It makes no claim to discovering the interaction.

3. Methodology

3.1 Four diagnostic checks

The runs used CPython 3.11.9, Black 25.1.0 and Ruff 0.11.13 on Windows. Formatting targeted Python 3.11 with line length 88 and preview disabled. Ruff selected F401 (unused import), F821 (undefined name) and F841 (unused local variable), with fixes disabled [6]. These are historical pins, not current-release recommendations. Analyzed programs were never executed.

Pass	Source	Suppression comments
A	Original	Respected
B	Original	Ignored
C	Formatted	Respected
D	Formatted	Ignored

Passes B and D reveal warnings hidden by `noqa`. Within one snapshot, the difference between underlying and visible diagnostics identifies suppression. The visible output must be a multiset subset of the underlying output. Rule, message and exact range identify a warning for this within-source subtraction; those coordinates cannot identify it across formatting.

3.2 Syntax and warning correspondence

The first attribution gate compares abstract syntax trees (ASTs), parsed with type comments enabled and ordinary location attributes omitted. Unequal dumps stop suppression attribution. This catches changes in parsed structure, but it is not a proof of runtime equivalence. In particular, `TypeIgnore.lineno` remains an AST field and can make otherwise matching dumps unequal.

Across snapshots, the identity key is the rule plus an AST field/index path. The matcher also requires the coordinate-free node content and construct/diagnostic tokens to agree. It supports import aliases, load Names and store Names for the selected rules. Only complete, unique correspondence with unchanged underlying warnings permits a visibility judgment: visible to suppressed is Newly hidden, and suppressed to visible is Newly exposed. Preserved, Mixed, Ambiguous, Diagnostic changed, AST changed and Execution failure remain distinct outcomes.

3.3 Inputs

The controlled design has 25 templates and 8 directive modes, giving 200 inputs. Modes include absent, standalone, inline specific, inline blanket, multicode and file-level directives. Treatments intentionally reuse executable structure. The source specification and validation were fixed before the main run; the examples were constructed to exercise formatting behavior rather than sampled from typical Python code.

The natural sample has 40 files from pinned Requests, Flask and pytest commits. Eligibility required UTF-8 Python under the study grammar, a genuine `noqa` comment, the documented size cap and generated/vendored exclusions. Seeded hash ranking selected files without using their results. Primary linting used standardized settings and an artificial filename. Separate checks used archived Ruff configuration and original paths where available; they did not reproduce full project CI.

3.4 Analysis and supplementary review

Both formatters received each original input, so their outcomes are paired. Treatments from one template also share structure. Whole-cluster resampling keeps those treatments and the formatter pairing together [7]; family-incidence Wilson calculations ask whether a family has any drift [8]. Detailed interval choices followed the earlier outcome counts, making the analysis exploratory.

A later review revisited every Ambiguous comparison using literal source occurrences, saved diagnostics and comment-only interventions. It proposed Preserved for all 32. A separate calculation applies those proposals; the automatic labels, original analysis and matcher are unchanged.

AI tools assisted implementation, debugging, controlled input generation, analysis scripts and writing, including this editorial revision. Measurements came from executed tools, with deterministic input and consistency checks. The supplementary review was also AI-assisted, not independent human adjudication. Natural source was collected without study-generated edits; its authors' AI use is unknown.

4. Results

4.1 Controlled comparisons

Black had 68 confirmed suppression changes and Ruff 35 on the same 200 inputs: 400 comparisons in total. These account for 34.0% and 17.5% of scheduled comparisons, respectively. The denominators include the unresolved cases listed below. No controlled comparison had an execution failure, AST change, underlying-diagnostic change or mixed drift.

Outcome	Original Black	Original Ruff	Reviewed Black	Reviewed Ruff
Preserved	116	149	132	165
Newly hidden	50	14	50	14
Newly exposed	18	21	18	21
Ambiguous	16	16	0	0

Each column contains 200 comparisons. The automatic columns keep the original labels. The reviewed columns apply only the supplementary proposals; confirmed-change counts stay the same. Figure 1 shows both versions.

The automatic comparison resolves 184 input pairs. On that common subset, drift is 36.96% for Black and 19.02% for Ruff, a difference of 17.93 percentage points. Accepting the review expands the denominator to 200 pairs and gives 16.50 points. The difference changes because the added pairs are reviewed as no drift for both formatters, not because more changes were found.

Paired drift status	Ruff no drift	Ruff drift
Black no drift	132	0
Black drift	33	35

The table uses the supplementary 200 pairs. A shared drift entry means both formatters changed visibility; it does not require the same direction of change.

4.2 Ambiguity and uncertainty

Two template families account for the ambiguity. In 16 f-string comparisons, original and formatted bytes are identical, but the embedded identifier does not align with a complete Python 3.11 token. In 16 exception-binding comparisons, only assignment spacing changes. The latter warning is attached to the string-valued `ExceptHandler.name`, outside the matcher's store-Name policy.

The separate review found no visibility transition in these cases, checking 128 saved lint replays and 64 comment interventions. Its human-review flags remain, particularly for identifying the same exception binding. These proposals do not constitute a validated extension of the matcher.

Cluster model	Original difference interval (pp)	Reviewed difference interval (pp)
Template families	8.70 to 28.26	8.00 to 25.50
Syntax categories	5.92 to 32.14	4.89 to 30.29

Each exploratory 95% model interval uses 20000 paired whole-cluster draws. The original analysis has 23 complete families and 12 categories; the reviewed analysis has 25 and 13. Its intervals were recalculated with the new denominator. Figure 2 shows the reviewed profiles. Original unresolved-outcome bounds and family-level Wilson results remain separate. None of these intervals establishes ecosystem coverage.

4.3 Source patterns

The source pairs show two concrete mechanisms. Black collapsed a short list whose first item had a specific `noqa` comment, leaving both names on the covered line. The second name's warning became hidden. Ruff kept that list expanded. With semicolon-separated assignments, both formatters split the line and exposed a warning previously covered by the trailing comment.

The pattern tables describe the fixed input/treatment grid, whose cells reuse templates. Figure 3 groups drift by category and placement. Its boundary cells use the review; the original figure keeps those cells unresolved.

Text changed in 133 Black comparisons and 118 Ruff comparisons. Of these, 57 and 75 were automatically Preserved; the reviewed counts are 65 and 83. Reformatting text and changing suppression are different events.

4.4 Natural-source comparisons

The 80 primary comparisons of 40 repository files produced no confirmed suppression change. The supplementary ambiguity review does not alter these results.

Repository	Files tested	Black Preserved / AST excluded	Ruff Preserved / AST excluded
Requests	7	7 / 0	7 / 0
Flask	13	12 / 1	13 / 0
pytest	20	18 / 2	19 / 1

Only 15 outputs changed; 65 stayed identical. Of the original files, 22 had no underlying selected-rule warnings, corresponding to 44 comparisons. A natural `noqa` comment can target rules outside this study, so eligibility alone did not guarantee relevant warnings. Original Figure 4 shows that diagnostic relevance.

The 4 AST exclusions came solely from shifted `TypeIgnore.lineno` fields. Zeroing those fields in a secondary investigation made the dumps equal, but the primary labels stay excluded. Native-configuration checks completed 66 comparisons with the same outcome classifications as the primary checks. Another 14 Requests records report missing configuration; they are statuses, not executed native comparisons.

The first natural repeat sample happened to contain only unchanged outputs. Phase 7 later repeated all 80 primary comparisons, including all 15 changed outputs, and matched the originals. This extends repeatability coverage while leaving the independent sample unchanged.

5. Discussion

Ruff had fewer suppression changes overall in this controlled suite, but fewer total changes did not mean fewer exposed warnings. Hiding and exposure need to be read separately. The comparison is about these inputs, settings and tool versions, not a general ranking of the formatters.

In the list and semicolon pairs, the underlying warning still belongs to the same source occurrence. What changes is whether the directive covers it. That distinction is the reason for checking correspondence before visibility. The experiment does not judge whether the warning is useful or whether a formatter should adopt a particular comment policy.

The natural sample offers fewer opportunities to observe those mechanisms: most outputs were unchanged and many selected warning sets were empty. Prior formatting could be one explanation for unchanged text, but repository formatting history was not measured here. A follow-up could define a separate stratum with relevant warnings and changed formatter output; its rate would describe that stratum, not all Python files.

The review also separates two matcher limitations from actual visibility changes. A wider matcher would need new anchors for those constructs, followed by testing on independently checked held-out examples. Resolving these saved cases alone cannot establish how that extension would behave elsewhere.

6. Limitations

The templates deliberately stress selected layouts, and directive variants share code. Cluster resampling preserves that within-group dependence, but independence and exchangeability between authored clusters remain assumptions. Interval choices were made after the earlier counts were known. Reviewed intervals are also conditional on the proposed labels and omit review error.

The repository sample is limited to a few historical projects chosen for feasibility. Its UTF-8, grammar, size, generated/vendored and natural-comment filters define a narrow frame. Empty warning sets and unchanged outputs restrict the negative finding further. The original analysis reports each archive's eligible population and completion bounds separately.

The AST gate and supported warning anchors limit attribution. AST equality does not prove runtime equivalence, and unsupported correspondence remains uncertain. The same AI-assisted process developed implementation, examples, tests and interpretation. Controls and repeated runs can expose inconsistencies, but they do not supply independently adjudicated held-out labels. Precision, recall and F1 therefore remain unmeasured.

Testing covered the pinned Windows/Python environment. The artificial primary filename differs from real project context, and native checks restore only part of that context. Other machines, operating systems, releases, diagnostic families and full project CI were not evaluated.

7. Conclusion

On the controlled inputs, formatting changed which Ruff warnings were suppressed while passing the strict AST gate. The repository sample produced no confirmed change. The later review supports Preserved proposals for the original ambiguities and leaves their automatic labels intact.

The next evaluation should establish correspondence accuracy on independently reviewed held-out pairs. A larger natural sample with relevant warnings and changed output would address a different gap. Both are needed before presenting this implementation as a developer check with measured error rates.

8. References

[1] Huimin Hu, Yingying Wang, Julia Rubin, and Michael Pradel. 2025. An Empirical Study of Suppressed Static Analysis Warnings. Proceedings of the ACM on Software Engineering 2(FSE), Article FSE014. DOI. Published author text.

[2] Junjie Li and Jinqiu Yang. 2024. Tracking the Evolution of Static Code Warnings: The State-of-the-Art and a Better Approach. IEEE Transactions on Software Engineering 50(3), 534-550. DOI. Author text.

[3] William M. McKeeman. 1998. Differential Testing for Software. Digital Technical Journal 10(1), 100-107. Original journal.

[4] Black contributors. Frequently Asked Questions, version 25.1.0. Versioned source.

[5] Astral contributors. Known Deviations from Black, Ruff 0.11.13. Versioned source.

[6] Astral contributors. Ruff linter and formatter documentation. Current pages provide background; retained pinned help and actual commands govern the experiment.

[7] StataCorp. Bootstrap resampling reference manual, cluster resampling. Manual. The calculation here used NumPy, not Stata.

[8] NIST/SEMATECH. e-Handbook of Statistical Methods, confidence limits for a proportion. Wilson method.

Data and reproduction

The reproduction guide covers the pinned environment and fresh output paths. Primary data and automatic metrics are under `datasets` and `reports/phase5/analysis`. The ambiguity audit records source occurrences, diagnostics and judgment reasons. Reviewed metrics and draws are under `reports/publication-phase-a/analysis`; the public document claim ledger is under `reports/publication-phase-b`.

The original report and its PDF remain frozen. In this public copy, execution metadata is anonymized and natural trial source uses lossless, noticed containers. Full upstream archives and unrelated assets are excluded; snapshot notes explain the reconstruction limits. MIT covers author-owned code and documentation, while upstream terms remain separate.